

Linux-Debian Généralités

Site:	Elearning Eric M
Cours:	ISCE 25-27 Bloc 1-2
Livre:	Linux-Debian Généralités

Imprimé par:	Eric MALETRAS
Date:	mercredi 10 juin 2026, 17:07

Table des matières

- 1. Introduction**
- 2. Les distributions Linux**
- 3. Les environnements de bureau**
- 4. L'arborescence et le système de fichiers**
- 5. Les commandes de base du système de fichiers**
- 6. Affichage et édition de fichiers**
- 7. La gestion des droits d'accès**
- 8. Modifier propriétaires et permissions**
- 9. Lire les droits : ls -l et la notation octale**
- 10. Maintenir son système : apt**
- 11. Installer hors dépôts : wget et tar**
- 12. Récapitulatif**

1. Introduction

BTS SIO SISR · SYSTÈMES LINUX

Linux / Debian — Les fondamentaux

Référentiel BTS SIO SISR

Blocs de compétences :

- **Bloc 2** - Administration des systèmes et des réseaux (E6)
- **Bloc 3** - Cybersécurité des services informatiques (E7)

Compétences :

- **B2.2** - Installer, tester et déployer une solution d'infrastructure
- **B2.3** - Exploiter, dépanner et superviser une solution d'infrastructure
- **B3.1** - Protéger les données à caractère personnel (gestion des droits d'accès)

Savoirs associés : systèmes d'exploitation libres, distributions GNU/Linux, arborescence et système de fichiers, ligne de commande, permissions Unix, gestion des paquets.

Pourquoi cette leçon ?

Linux équipe la grande majorité des serveurs en entreprise (web, supervision, virtualisation, conteneurs). En SISR, vous administrerez quotidiennement des serveurs Debian en ligne de commande, sans interface graphique : il est donc indispensable de maîtriser l'arborescence, les commandes de base, les droits d'accès et la gestion des paquets avant d'aborder les services d'infrastructure (Apache, MariaDB, GLPI, Zabbix...).

Objectifs

Comprendre l'origine et la philosophie de GNU/Linux, choisir une distribution adaptée, naviguer dans le système de fichiers, gérer les permissions et maintenir un système Debian à jour.

Durée

Environ 4 heures (lecture + manipulations sur VM Debian 12).

Prérequis

Notions de base sur les systèmes d'exploitation. Une VM Debian 12 fonctionnelle (VMware Workstation ou Proxmox).

Aux origines : Unix et le projet GNU

À l'origine de quasiment tous les systèmes d'exploitation modernes se trouve **Unix**, créé en **1969** aux Bell Labs. Unix était beaucoup plus puissant que MS-DOS (1981, l'ancêtre de Windows), mais aussi beaucoup plus compliqué à utiliser.

En **1984**, Richard Stallman, alors chercheur en intelligence artificielle au MIT, lance le **projet GNU**. Son objectif : créer un nouveau système d'exploitation fonctionnant comme Unix (les commandes restant les mêmes), mais gratuit et libre.

Logiciel libre

Un programme **libre** est un programme dont on a accès aux sources et que l'on peut donc **copier, modifier et redistribuer**. Attention : libre ne signifie pas nécessairement gratuit, même si c'est souvent le cas.

Linux côté serveur : une logique différente de Windows

Sous Linux, il n'existe pas de version spécifiquement « serveur » comme sous Windows. Vous ne trouverez pas de distribution Linux embarquant l'ensemble des rôles serveurs comme Windows Server le fait avec DNS, [DHCP](#), AD, WDS...

Avec Linux, il vous faudra systématiquement récupérer les applications serveur sur [Internet](#), via les dépôts de la distribution. Et pour

un même rôle — un serveur [DHCP](#) par exemple — vous aurez le choix entre plusieurs applications concurrentes.

 À retenir

Unix (1969) → projet GNU (1984, Stallman) → noyau Linux (1991, Torvalds). Un système GNU/Linux = noyau Linux + outils GNU + logiciels assemblés en distribution.

2. Les distributions Linux

LINUX / DEBIAN · CHAPITRE 2

Les distributions Linux

Noyau ou distribution ?

Quand on parle de « Linux », on parle en fait du **noyau** (*kernel*) de notre OS. Mais on n'installe jamais — du moins très rarement — seulement le noyau : on installe plutôt une **distribution Linux**.

Une distribution Linux, appelée aussi distribution **GNU/Linux** lorsqu'elle contient les logiciels du projet GNU, est un ensemble cohérent de logiciels, la plupart étant des logiciels libres, assemblés autour du noyau Linux et formant un système d'exploitation pleinement opérationnel.

Ce qui différencie les distributions

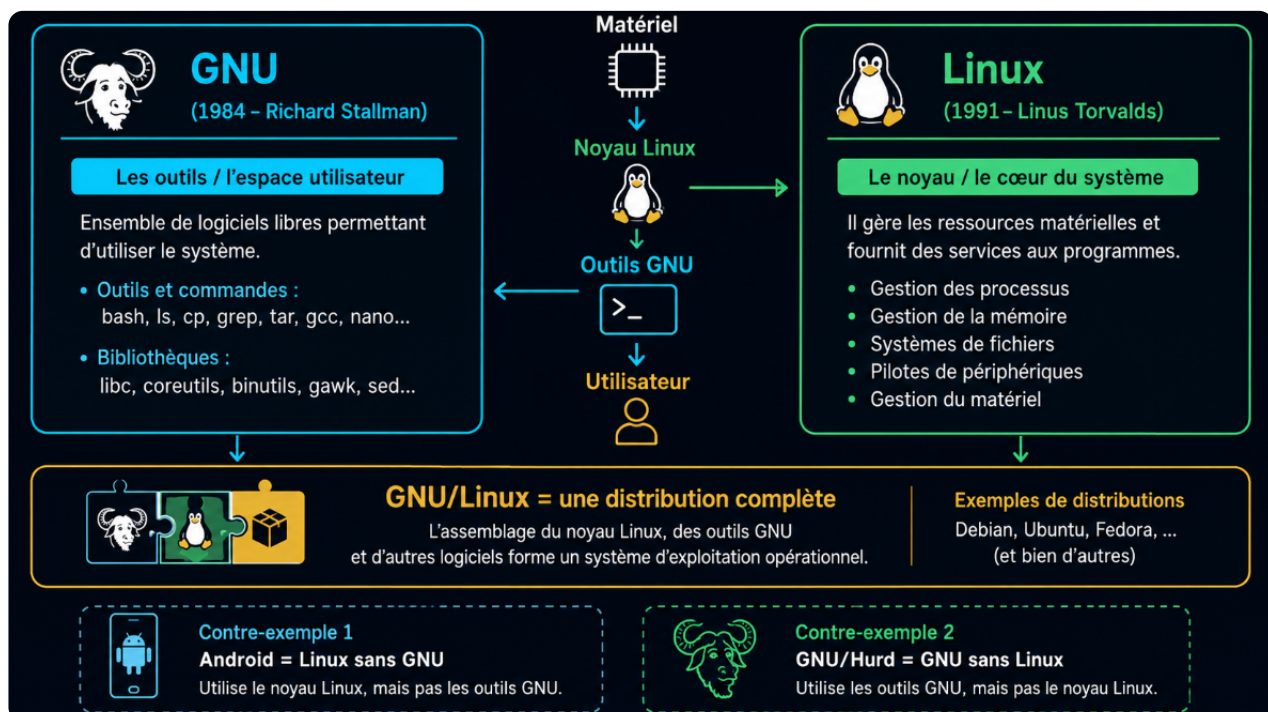
Il existe une très grande variété de distributions Linux, chacune ayant des objectifs et une philosophie particulière. Les éléments les différenciant principalement sont :

- la **convivialité** (facilité de mise en œuvre),
- l'**intégration** (taille du parc de logiciels validés distribués),
- la **notoriété** (communauté informative pour résoudre les problèmes),
- leur **fréquence de mise à jour**,
- leur **gestion des paquets** et le **mainteneur** de la distribution (généralement une entreprise ou une communauté).

Leur point commun est le noyau Linux et un certain nombre de commandes Unix.

💡 GNU et Linux sont indépendants

Les parties GNU et Linux d'un système d'exploitation sont indépendantes : on trouve aussi bien des systèmes avec Linux et sans GNU, comme **Android**, que des systèmes GNU sans Linux, comme **GNU/Hurd**.



Les grandes familles de distributions

Il existe plusieurs centaines de distributions, chacune ayant ses particularités :

- Les distributions **à usage spécifique**, automatisant et facilitant l'installation des composants nécessaires pour réaliser par exemple : pare-feu, routeur, grappe de calcul, édition multimédia...
- Les distributions pouvant être installées sur un **grand nombre de matériels spécifiques** (grâce au support de nombreuses architectures).
- Les distributions **généralistes**, dont les plus populaires sont : Debian, Gentoo, Mageia, Fedora, Slackware, openSUSE ou encore Ubuntu, permettant des usages variés.
- Les distributions orientées exclusivement vers l'**entreprise**, avec un contrat de support annuel (à base de souscription par machine), par exemple Red Hat Enterprise Linux, Ubuntu Long Term Support et SUSE Linux Enterprise.

La maintenance d'une distribution peut être assurée par une **entreprise** (cas de Red Hat Enterprise Linux, SUSE Linux Enterprise...) ou par une **communauté** (cas de Debian, Mageia, Gentoo, Fedora, Ubuntu, Slackware...). Certaines communautés peuvent aussi avoir comme mainteneur principal une entreprise : c'est le cas de Fedora dont Red Hat est le premier sponsor, d'Ubuntu par Canonical ou encore d'openSUSE par Novell.

⚠ Libre ne veut pas dire gratuit

Les logiciels du projet GNU sont libres, ils utilisent tous la licence GPLv3. Linux, quant à lui, est partiellement libre, sous licence GPLv2, car il contient aussi une quantité importante de code qui n'est pas libre.

Une majorité des logiciels contenus dans les dépôts des systèmes GNU/Linux sont libres, mais libre ne veut pas dire gratuit, même si les logiciels libres sont généralement distribués gratuitement. Ainsi, lorsque l'on achète une distribution GNU/Linux, le prix payé est celui du média, de la documentation incluse et du travail effectué pour assembler les logiciels en un tout cohérent. Pour se conformer aux exigences des licences, les éditeurs acceptent de mettre à disposition les sources des logiciels sans frais supplémentaires.

Un peu d'histoire

La première distribution, apparue en **1992**, était assemblée sur quelques dizaines de disquettes. En raison de la très forte croissance de GNU/Linux, une distribution actuelle peut occuper de quelques mégaoctets (pour être installée sur une clé USB par exemple) à plusieurs gigaoctets.

Avant l'existence des distributions, les utilisateurs de GNU/Linux devaient composer eux-mêmes leur système en réunissant tous les éléments nécessaires.

Les principales distributions

 Grand public

- **Ubuntu** : basée sur Debian, c'est une distribution commerciale orientée grand public, distribuée gratuitement par Canonical, qui édite des versions stables tous les six mois (maintenues neuf mois) et des versions « LTS » (maintenues plusieurs années) tous les deux ans. Différentes versions ou saveurs (*flavours*) se distinguent par leur environnement graphique (GNOME, KDE, Xfce, LXDE, Cinnamon, MATE).
- **Linux Mint** : distribution orientée grand public basée sur Ubuntu, conçue pour être facile d'installation et d'usage. Elle est aussi disponible avec une base Debian, alors dénommée LMDE (Linux Mint Debian Edition).
- **Fedora** : distribution grand public communautaire supervisée par Red Hat, basée sur le système de gestion de paquets RPM. Elle met l'accent sur la nouveauté : ses logiciels sont très fréquemment mis à jour. Elle suit le cycle de sortie de GNOME tous les six mois.
- **Manjaro** : distribution basée sur Arch Linux. Elle fonctionne selon le modèle *rolling release*, c'est-à-dire qu'elle se met à jour en permanence sans changement de version du système.
- **openSUSE** : distribution communautaire destinée tant à un usage grand public que professionnel. Elle est sponsorisée principalement par SUSE qui l'utilise comme base pour ses solutions commerciales. Distribution indépendante réputée pour ses outils de configuration et sa stabilité.
- **Solus** : distribution indépendante orientée grand public qui utilise le modèle de rolling release.

 Entreprise ou communauté

- **Red Hat Enterprise Linux (RHEL)** : distribution commerciale largement répandue dans les entreprises (surtout aux États-Unis).
- **Arch Linux** : distribution communautaire sans versions, en mise à jour permanente. Elle dispose toujours des dernières versions des logiciels disponibles, grâce à une communauté de développeurs très active.
- **Debian** : se distingue par le très grand nombre d'architectures supportées, son importante logithèque et son cycle de développement relativement long, gage d'une grande stabilité.
- **SUSE Linux Enterprise** : distribution issue d'openSUSE. Elle utilise le format de paquets RPM (RPM Package Manager) développé par Red Hat. Distribution réputée pour ses outils de configuration et sa stabilité.
- **Gentoo** : distribution caractérisée par sa gestion des paquets à la manière des ports BSD, effectuant généralement la compilation des logiciels (X, OpenOffice, etc.) sur la machine de l'utilisateur. Elle est destinée aux utilisateurs avancés, aux développeurs et aux passionnés.
- **Slackware** : une des plus anciennes distributions existantes, particulièrement adaptée aux utilisations serveur.

 Pour aller plus loin

La plupart des autres distributions sont des *forks* (copies modifiées) de ces distributions historiques. Pour en avoir un aperçu, vous pouvez consulter la [généalogie complète des distributions Linux](#).

 À retenir

Linux = le noyau ; distribution = noyau + logiciels assemblés. En BTS SISR, nous utiliserons **Debian**, référence des serveurs en entreprise pour sa stabilité, et base d'Ubuntu.

3. Les environnements de bureau

LINUX / DEBIAN · CHAPITRE 3

Les environnements de bureau

Qu'est-ce qu'un environnement de bureau ?

Un **environnement de bureau** (*desktop environment*) est un logiciel qui permet de manipuler l'ordinateur à travers une interface utilisateur en **mode graphique**.

Les premiers environnements de bureau sont apparus dans les années 80. Windows est apparu en **1985**, les premiers environnements pour Unix vers **1993**, et les environnements sous Linux sont devenus matures avec **GNOME et KDE** au début des années 2010.

Les deux grandes familles

Sous Linux, il existe deux grandes familles d'environnements de bureau, suivant leur **bibliothèque graphique**, présentes sur la plupart des distributions Linux :

Famille GTK

- **GNOME**
- Unity
- Cinnamon
- MATE
- Xfce
- LXDE

Famille Qt

- **KDE**
- LXQt
- Razor-qt
- Elokab

Et d'autres encore...

Il existe aussi des environnements n'appartenant à aucune de ces deux familles, comme **Enlightenment (E17)** ou **Étoilé**.

En administration serveur : pas de bureau !

La plupart du temps, nous n'utiliserons **pas d'environnement de bureau** sur nos serveurs, pour gagner en taille disque et en ressources (RAM, CPU). Un serveur Debian s'administre en ligne de commande, localement ou à distance via SSH. C'est aussi une bonne pratique de sécurité : moins de logiciels installés = moins de surface d'attaque.

À retenir

Environnement de bureau = interface graphique, optionnelle sous Linux. Deux familles principales selon la bibliothèque graphique : GTK (GNOME, Xfce...) et Qt (KDE...). Sur serveur, on s'en passe.

4. L'arborescence et le système de fichiers

LINUX / DEBIAN · CHAPITRE 4

L'arborescence et le système de fichiers

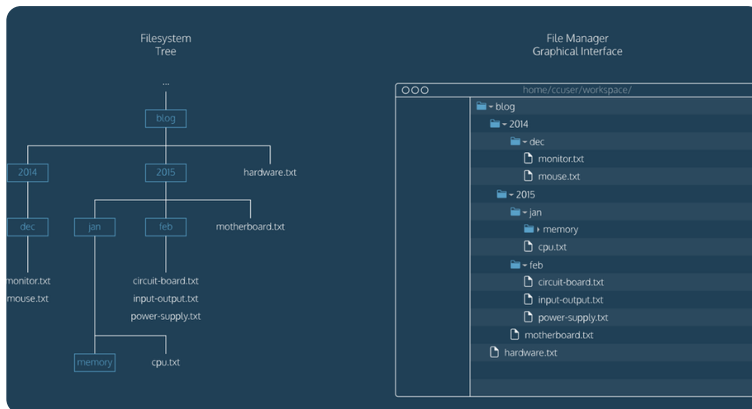
La ligne de commande

La **ligne de commande** est une interface textuelle pour piloter le système d'exploitation d'un ordinateur. Pour accéder à la ligne de commande, on utilise un **terminal**, appelé également **console**.

Le système de fichiers

Les fichiers et répertoires d'un ordinateur forment une arborescence appelée **système de fichiers** (*filesystem*).

Le point de départ du système de fichiers est un répertoire appelé **racine**, notée `/` (slash) sous Linux (l'équivalent du `C:` sous Windows). Chaque répertoire peut contenir des fichiers ainsi que des sous-répertoires.



Une structure de dossiers stricte

Linux fonctionne sur une structure de répertoires extrêmement stricte et normalisée. Les principaux répertoires à la racine sont :

Répertoire	Contenu
<code>/bin</code>	Les exécutables, les commandes (ainsi que <code>/usr/bin</code> , <code>/usr/local/bin</code>)
<code>/sbin</code>	Les exécutables réservés à root (ainsi que <code>/usr/sbin</code> , <code>/usr/local/sbin</code>)
<code>/etc</code>	Les fichiers de configuration
<code>/home</code>	Les répertoires de travail des utilisateurs
<code>/lib</code>	Les bibliothèques partagées
<code>/dev</code>	Les périphériques
<code>/usr</code>	Les autres programmes et leurs fichiers annexes liés aux utilisateurs
<code>/var</code>	Les données variables (logs, sites web, bases de données...)

Les chemins : absolus et relatifs

L'emplacement de chaque ressource (fichier ou répertoire) dans le système de fichiers est appelé son **chemin**. Dans un chemin Linux, le séparateur entre deux répertoires est le caractère `/`.

On distingue deux types de chemins :

- Un chemin **absolu** identifie une ressource en commençant à la racine de l'arborescence, avec le caractère `/`. Un chemin absolu ne dépend pas du répertoire courant et est donc valide partout.
`/home/baptiste/hello.txt` et `/etc/apache/httpd.conf` sont des exemples de chemins absolus.
- Un chemin **relatif** identifie une ressource à partir du répertoire courant. Il dépend donc du répertoire courant et n'est pas valide partout.
`../marc/adresses.txt` et `documents/cours/si1.pdf` (sans `/` au début !) sont des exemples de chemins relatifs.
En chemin relatif, si l'on veut « remonter » dans l'arborescence par rapport à sa position, on utilisera les `..`

Le répertoire personnel

Sous Linux, chaque utilisateur (sauf root) dispose d'un **répertoire personnel** à son nom, situé dans `/home`. Par exemple, le répertoire personnel de l'utilisateur robert est `/home/robert`.

Le chemin absolu du répertoire personnel peut s'écrire de manière abrégée avec le caractère `~` (tilde). Par exemple, le chemin `~/music/` pour l'utilisateur robert correspond au chemin absolu `/home/robert/music/`.

💡 Et root ?

Le répertoire personnel du super-utilisateur **root** n'est pas dans `/home` : c'est `/root`, directement à la racine.

✅ À retenir

Racine = `/`. Chemin absolu : commence par `/`, valide partout. Chemin relatif : dépend du répertoire courant, `..` pour remonter.
Répertoire personnel : `/home/utilisateur`, abrégé `~`. Configuration dans `/etc`, données variables dans `/var`.

5. Les commandes de base du système de fichiers

LINUX / DEBIAN · CHAPITRE 5

Les commandes de base du système de fichiers

Se repérer et naviguer

- `pwd` (*print working directory*) affiche le chemin du répertoire courant.
- `ls` (*list*) affiche le contenu du répertoire courant.
 - `ls -a` affiche également les fichiers cachés, qui commencent par un `.` sous Linux.
 - `ls -l` affiche des informations supplémentaires, comme la date et la taille des fichiers.
 - `ls -t` trie les fichiers par ordre de dernière modification.
- `cd` (*change directory*) permet de se déplacer dans le système de fichiers en changeant de répertoire courant.
 - `cd monrep` fait du répertoire monrep le répertoire courant.
 - `cd ..` permet de remonter d'un niveau dans l'arborescence.
 - `cd /` permet de revenir à la racine de l'arborescence.
 - `cd` (sans argument) permet de revenir à la racine du répertoire personnel.

Créer des répertoires et des fichiers

- `mkdir` (*make directory*) crée un nouveau répertoire.
`mkdir monrep` crée le répertoire monrep dans le répertoire courant.
- `touch` crée un nouveau fichier (vide) ou met à jour la date de modification d'un fichier existant.
`touch fic1.txt` crée un fichier vide fic1.txt dans le répertoire courant.

Copier, déplacer, supprimer

- `cp` (*copy*) copie des fichiers ou des répertoires.
`cp fic1.txt monrep/` copie le fichier fic1.txt dans le répertoire monrep.
`cp fic1.txt fic2.txt` duplique le fichier fic1.txt sous le nom fic2.txt.
- `mv` (*move*) déplace ou renomme des fichiers ou des répertoires.
`mv fic1.txt monrep/` déplace le fichier fic1.txt dans le répertoire monrep.
`mv fic1.txt fic2.txt` renomme le fichier fic1.txt en fic2.txt.
- `rm` (*remove*) supprime des fichiers. `rm -r` supprime des répertoires.
`rm fic1.txt` supprime le fichier fic1.txt.
`rm -r monrep` supprime le répertoire monrep ainsi que tout son contenu.

⚠ Pas de corbeille en ligne de commande !

Une suppression avec `rm` est **définitive et immédiate** : il n'y a pas de corbeille, pas de confirmation par défaut, pas de retour en arrière. Relisez toujours votre commande avant de valider, surtout en root.

Le caractère générique

Le caractère générique `*` (*wildcard*) permet de remplacer une partie d'un nom de fichier ou de répertoire. On l'utilise pour appliquer une commande à plusieurs éléments.

- `cp f*.txt monrep/` copie dans le répertoire monrep tous les fichiers dont le nom commence par un `f` et finit par `.txt`.

- `rm *` supprime tous les fichiers du répertoire courant.

✓ À retenir

Navigation : `pwd` (où suis-je ?), `ls` (qu'y a-t-il ici ?), `cd` (j'y vais). Création : `mkdir`, `touch`. Manipulation : `cp`, `mv`, `rm`. Le `*` généralise une commande à plusieurs fichiers — à manier avec précaution avec `rm`.

6. Affichage et édition de fichiers

LINUX / DEBIAN · CHAPITRE 6

Affichage et édition de fichiers

Afficher du texte et des fichiers

- `echo` affiche un texte.
`echo Bonjour Monde` affiche le texte « Bonjour Monde ».
- `cat` permet (entre autres) d'afficher le contenu d'un fichier.
`cat monfic` affiche le contenu du fichier monfic.
- `clear` réinitialise le contenu de la console.

Les redirections

Le caractère `>` permet de rediriger la sortie d'une commande vers un fichier **en écrasant** son contenu actuel.

Le caractère `>>` redirige la sortie d'une commande vers un fichier **en l'ajoutant à la fin** de son contenu actuel.

- `echo Bonjour Monde > bonjour.txt` remplace le contenu du fichier bonjour.txt par le texte « Bonjour Monde ».
- `echo Bonjour Monde >> bonjour.txt` ajoute le texte « Bonjour Monde » à la fin du fichier bonjour.txt.

⚠ Un seul chevron écrase tout

Confondre `>` et `>>` sur un fichier de configuration peut détruire son contenu sans avertissement. En cas de doute, utilisez `>>` ou faites une copie de sauvegarde avant (`cp fichier fichier.bak`).

Pas d'association extension → application

Sous Linux en mode CLI, vous ne pouvez pas double-cliquer sur un fichier qui, suivant son extension, va s'ouvrir avec le bon logiciel. Sous Windows, un document Excel (.xls) va s'ouvrir avec Excel, un document HTML (.html) va s'ouvrir avec votre navigateur [Internet...](#)

Sous Linux, la notion d'extension liée à une application n'existe pas. Vous devrez donc systématiquement spécifier avec quelle application vous désirez ouvrir un fichier. Pour ce faire, votre ligne de commande commencera toujours par l'application, puis seulement le nom (avec le chemin si besoin) de votre fichier :

```
nano /home/robert/Documents/fichier1.txt
```

L'éditeur de texte Nano

`nano` lance un éditeur de texte nommé Nano. Il dispose de plusieurs raccourcis clavier, dont :

- `Ctrl+O` pour sauvegarder un fichier.
- `Ctrl+X` pour quitter l'éditeur.

Il existe d'autres éditeurs de texte (tels que **Vim** ou **Vi**), mais Nano est fourni par défaut sur Debian.

✓ À retenir

Affichage : `echo` (texte), `cat` (fichier). Redirections : `>` écrase, `>>` ajoute. Sous Linux, pas d'association extension/application : on précise toujours l'application avant le fichier. Édition : `nano fichier`, puis `Ctrl+O` pour sauver, `Ctrl+X` pour quitter.

7. La gestion des droits d'accès

LINUX / DEBIAN · CHAPITRE 7

La gestion des droits d'accès

Un héritage d'Unix

Le système de droits d'accès de Linux hérite directement du système **UNIX**. La gestion des permissions se fait sur trois niveaux d'« utilisateurs » :

- Le **propriétaire** (`u` pour *user*), par défaut le créateur du fichier/dossier ;
- Le **groupe propriétaire** (`g` pour *group*) du fichier/dossier, ensemble d'utilisateurs ayant des droits spécifiques sur le fichier/dossier ;
- Les **autres** (`o` pour *others*), n'étant ni propriétaire, ni membres du groupe propriétaire.

Les trois permissions

Pour chacun de ces trois niveaux, il est possible d'attribuer des permissions différentes, qui sont :

- La **lecture** (`r` pour *read*) : permet de lire un fichier ou d'afficher le contenu d'un dossier ;
- L'**écriture** (`w` pour *write*) : permet de modifier le contenu d'un fichier. Au niveau d'un dossier, elle permet d'ajouter, modifier, renommer ou supprimer un fichier ;
- L'**exécution** (`x` pour *execute*) : permet de pouvoir exécuter un fichier programme. Au niveau d'un dossier, elle permet d'y entrer (`cd`).

💡 Configuration courante

Souvent, les permissions sont de la sorte :

- `u` : `rw`x — le propriétaire a tous les droits ;
- `g` : `rx` — le groupe peut lire et exécuter/traverser ;
- `o` : `r` ou `rx` — les autres peuvent au mieux lire.

Deux familles de commandes pour modifier

On peut modifier les permissions à deux niveaux :

- Au niveau des **utilisateurs** (qui possède le fichier ?) avec les commandes `chown` et `chgrp` ;
- Au niveau des **permissions** (que peut-on en faire ?) avec la commande `chmod`.

Ces commandes sont détaillées dans le chapitre suivant.

✅ À retenir

Trois niveaux : `u` (propriétaire), `g` (groupe), `o` (autres). Trois permissions : `r` (lecture), `w` (écriture), `x` (exécution). Qui possède → `chown` / `chgrp` ; ce qu'on peut faire → `chmod`.

8. Modifier propriétaires et permissions

LINUX / DEBIAN · CHAPITRE 8

Modifier propriétaires et permissions

Changer le propriétaire : chown et chgrp

Modification du propriétaire et/ou du groupe propriétaire :

- Modification du **propriétaire** :

```
chown nom_propriétaire nom_du_fichier
```

- Modification du **propriétaire et du groupe propriétaire** :

```
chown nom_propriétaire:nom_groupe_propriétaire nom_du_fichier
```

- Modification du **groupe propriétaire** seul :

```
chgrp nom_groupe_propriétaire nom_du_fichier
```

💡 Récursivité

L'option `-R` applique le changement à un dossier **et à tout son contenu** (sous-dossiers et fichiers). Exemple classique lors du déploiement d'une application web :

```
chown www-data:www-data -R /var/www/glpi
```

donne le dossier glpi et tout son contenu à l'utilisateur `www-data` et au groupe `www-data` (le compte sous lequel tourne le serveur web Apache).

Changer les permissions : chmod

La modification des permissions via `chmod` est plus complexe. La syntaxe symbolique est :

```
chmod utilisateurs retrait/ajout permission fichier
```

C'est-à-dire : à **qui** (`u`, `g`, `o`, ou `a` pour tous), on **ajoute** (`+`) ou on **retire** (`-`) **quelle permission** (`r`, `w`, `x`), sur **quel fichier**.

Par exemple :

- Si l'on veut donner (`+`) au groupe propriétaire (`g`) le droit d'écriture (`w`) sur le fichier texte.txt :

```
chmod g+w texte.txt
```

- Si l'on veut retirer (`-`) le droit de lecture (`r`) à tous les autres (`o`) :

```
chmod o-r texte.txt
```

- Si l'on veut donner au propriétaire (`u`) les droits de lecture et d'écriture en une seule fois :

```
chmod u+rwx mon_fichier
```

⚠️ Qui a le droit de modifier ?

Seuls le **propriétaire du fichier** et **root** peuvent modifier ses permissions avec `chmod`. Quant à `chown`, il est réservé à **root** : un utilisateur ne peut pas « donner » ses fichiers à quelqu'un d'autre.

 À retenir

`chown user fichier` change le propriétaire ; `chown user:groupe fichier` change les deux ; `chgrp groupe fichier` change le groupe seul. `chmod qui+quoi fichier` ajuste les permissions (ex. `g+w` , `o-r`). L'option `-R` rend l'opération récursive.

9. Lire les droits : ls -l et la notation octale

LINUX / DEBIAN · CHAPITRE 9

Lire les droits : ls -l et la notation octale

Afficher les permissions avec ls -l

Pour afficher les permissions sur un dossier/fichier, on utilise la commande `ls`, qui liste les fichiers/dossiers dans le dossier actif, avec l'option `-l`.

Cette commande, `ls -l`, affichera le fichier avec un ensemble de **9 caractères** (trois triplets) qui représentent les 3 permissions par niveau d'utilisateur (propriétaire, groupe propriétaire, autres) :

- Le **premier triplet** correspond au propriétaire, le **second** au groupe et le **dernier** aux autres ;
- Pour chaque triplet, le premier caractère correspond à la permission de lecture (`r`), le second à l'écriture (`w`), le dernier à l'**exécution** (`x`).

Si l'utilisateur a la permission ad hoc, le caractère correspondant sera affiché, sinon vous aurez un `-`.

Exemples :

- `rwxr-x--x texte.txt` signifie : pour le propriétaire, les permissions de lecture, écriture, exécution ; pour le groupe, lecture et exécution ; pour les autres, juste exécution.
- `rwrxrw-x--x texte.txt` : tous les droits pour le propriétaire et le groupe, juste exécution pour les autres.

💡 Appliquer à tous d'un coup

`chmod -R a+rx mon_dossier` donne à **tous** les utilisateurs (`a` pour *all*) les droits en lecture et en exécution à tout ce que contient le dossier mon_dossier. Le `a` est facultatif : `chmod -R +rx mon_dossier` fonctionne tout aussi bien.

La notation octale

Il est possible d'écrire les permissions sous une forme numérique (**octale**) avec comme correspondance :

- `r` (4) : autorisation de lecture ;
- `w` (2) : autorisation d'écriture ;
- `x` (1) : autorisation d'exécution.

Il suffit d'indiquer pour chaque niveau (propriétaire, groupe, autres) la **somme** des permissions :

Droit	Valeur alphanumérique	Valeur octale
aucun droit	<code>---</code>	0
exécution seulement	<code>--x</code>	1
écriture seulement	<code>-w-</code>	2
écriture et exécution	<code>-wx</code>	3
lecture seulement	<code>r--</code>	4
lecture et exécution	<code>r-x</code>	5
lecture et écriture	<code>rw-</code>	6

Droit	Valeur alphanumérique	Valeur octale
tous les droits (lecture, écriture et exécution)	<code>rwX</code>	7

Les deux valeurs classiques

- `chmod 755 mon_dossier` donne au propriétaire tous les droits, aux membres du groupe et aux autres les droits de lecture et d'accès. C'est un droit utilisé traditionnellement sur les **répertoires**.
- `chmod 644 mon_fichier` donne au propriétaire les droits de modification et lecture, aux membres du groupe et aux autres uniquement les droits de lecture. C'est un droit utilisé traditionnellement sur les **fichiers**.

⚠ **777 : la fausse bonne idée**

`chmod 777` (tous les droits pour tout le monde) « règle » souvent un problème de permissions en TP... en ouvrant une faille de sécurité : n'importe quel utilisateur ou service compromis peut alors modifier ou exécuter le fichier. En production, on diagnostique le vrai besoin (propriétaire ? groupe ?) au lieu de tout ouvrir.

✅ **À retenir**

`ls -l` affiche 3 triplets `rwX` : propriétaire, groupe, autres. Octal : r=4, w=2, x=1, on additionne par triplet. Standards : **755** pour les répertoires, **644** pour les fichiers. **777** = à proscrire.

10. Maintenir son système : apt

LINUX / DEBIAN · CHAPITRE 10

Maintenir son système : apt

Le gestionnaire de paquets

La mise à jour d'un Debian sans interface graphique n'est pas automatique : elle doit être lancée manuellement via une application, `apt`, dont la commande historique est `apt-get` (mais `apt` tout court fonctionne aussi, et c'est aujourd'hui la forme recommandée en usage interactif).

Cette application sert aussi à installer un grand nombre d'applications, depuis les **dépôts** Debian : des serveurs hébergeant l'ensemble des paquets validés par l'équipe Debian.

La mise à jour en deux étapes

La mise à jour d'un Debian se fait en **deux étapes** :

1. Mise à jour de la base de données :

Votre système possède une base de données locale recensant l'ensemble des mises à jour et applications validées par l'équipe Debian. Cette base de données doit être mise à jour régulièrement via la commande :

```
apt update
```

2. Mise à jour du système et des applications :

Une fois votre base de données mise à jour, vous pourrez mettre à jour votre système via la commande :

```
apt upgrade
```

Vous pouvez cumuler les deux en une seule ligne :

```
apt update && apt upgrade
```

💡 **update ≠ upgrade**

Piège classique : `apt update` ne met **rien** à jour sur votre système — il rafraîchit seulement la **liste** de ce qui est disponible. C'est `apt upgrade` qui télécharge et installe réellement les nouvelles versions. D'où l'ordre impératif : d'abord `update`, ensuite `upgrade`.

Installer une application via apt

La méthode la plus utilisée pour ajouter une application sous Debian est aussi d'utiliser apt, via la commande :

```
apt install nom_du_programme
```

Exemple : `apt install apache2` installe le serveur web Apache, avec toutes ses dépendances, depuis les dépôts officiels.

⚠️ **Droits administrateur requis**

Les commandes apt modifient le système : elles doivent être exécutées en **root** (ou via `sudo` si configuré). Un utilisateur standard obtiendra une erreur « *Permission denied* ».

 À retenir

1 `apt update` rafraîchit la liste des paquets disponibles. 2 `apt upgrade` installe les mises à jour. En une ligne : `apt update && apt upgrade`. Installation : `apt install nom_du_programme`. Toujours en root.

11. Installer hors dépôts : wget et tar

LINUX / DEBIAN · CHAPITRE 11

Installer hors dépôts : wget et tar

Une autre méthode d'installation

Une autre méthode d'installation d'une application est de la télécharger sous format compressé via la commande `wget url_du_fichier`, puis de la décompresser avec la commande `tar nom_du_fichier`.

C'est la méthode utilisée lorsque l'application n'est pas disponible dans les dépôts Debian, ou lorsque l'on veut une version plus récente que celle des dépôts (cas typique : GLPI).

💡 wget n'est pas toujours présent

Si l'application `tar` est native sur Debian, il faudra en revanche installer `wget` au préalable : `apt install wget`.

La commande wget

Les options principales de wget sont :

- `wget -O NomDuFichierDeDestination UrlDuFichier` : télécharge en renommant le fichier (`-O` = *output*) ;
- `wget -P DossierDeDestination UrlDuFichier` : télécharge dans le dossier indiqué (`-P` = *prefix*).

La commande tar

Les options principales de tar sont :

- `-x` : extrait l'archive (*extract*) ;
- `-f` : utilise le fichier donné en paramètre (*file*) ;
- `-v` : active le mode verbeux (*verbose*, affiche les fichiers traités) ;
- Ajout du type de compression :
 - `-z` : compression Gzip (.tgz, .tar.gz) ;
 - `-j` : compression Bzip2 (.tar.bz2) ;
 - `-J` : compression Lzma/XZ (.tar.xz).

Sur les versions récentes de tar, le type de compression est détecté automatiquement à l'extraction : `tar -xvf archive.tgz` suffit.

Exemple complet : déployer GLPI

Pour installer GLPI dans le dossier `/var/www` :

```
# Téléchargement de l'archive dans /var/www
wget -P /var/www https://github.com/glpi-project/glpi/releases/download/10.0.6/glpi-10.0.6.tgz

# Extraction de l'archive (verbeux)
tar -xvf /var/www/glpi-10.0.6.tgz
```

⚠ Attention au répertoire d'extraction

`tar` extrait dans le **répertoire courant**, pas à côté de l'archive ! Dans l'exemple ci-dessus, il faut d'abord se placer dans `/var/www` (`cd /var/www`) avant d'extraire, ou utiliser l'option `-C /var/www` pour cibler le dossier de destination.

✅ À retenir

`wget -P dossier url` télécharge dans un dossier ; `wget -O nom url` renomme à la volée. `tar -xvf archive` extrait (x = extract, v = verbeux, f = fichier) dans le répertoire courant. Penser à `chown www-data:www-data -R` après extraction dans `/var/www` (chapitre 8).

12. Récapitulatif

LINUX / DEBIAN · CHAPITRE 12

Récapitulatif de la leçon

L'essentiel en 6 points

Histoire

Unix (1969) → projet GNU de Stallman (1984) → noyau Linux de Torvalds (1991). Un programme **libre** : sources accessibles, copiable, modifiable, redistribuable. Libre ≠ gratuit.

Distributions

Linux = le noyau ; on installe une **distribution** (noyau + outils GNU + logiciels). Grand public : Ubuntu, Mint, Fedora. Entreprise/serveur : Debian, RHEL, SUSE. Notre référence : **Debian**.

Bureau

Environnement graphique **optionnel**. Deux familles : GTK (GNOME, Xfce...) et Qt (KDE...). Sur serveur : pas de bureau, administration en CLI, gain de ressources et de sécurité.

Arborescence

Racine `/`. Configuration dans `/etc`, utilisateurs dans `/home`, données variables dans `/var`, exécutables dans `/bin` et `/sbin`. Chemin absolu : commence par `/` ; relatif : depuis le répertoire courant, `..` pour remonter, `~` = répertoire personnel.

Droits

3 niveaux : `u` / `g` / `o`. 3 permissions : `r` / `w` / `x`. Octal : `r=4`, `w=2`, `x=1`. Standards : **755** répertoires, **644** fichiers. **777** = à proscrire.

Maintenance

`apt update` (rafraîchit la liste) puis `apt upgrade` (installe). `apt install` pour les dépôts ; `wget` + `tar` pour le reste (GLPI...).

Mémo des commandes

Commande	Rôle	Exemple
<code>pwd</code>	Afficher le répertoire courant	<code>pwd</code>
<code>ls</code>	Lister le contenu (<code>-a</code> cachés, <code>-l</code> détails, <code>-t</code> tri par date)	<code>ls -l /var/www</code>
<code>cd</code>	Changer de répertoire (<code>..</code> remonter, <code>/</code> racine, seul = home)	<code>cd /etc</code>
<code>mkdir</code>	Créer un répertoire	<code>mkdir monrep</code>
<code>touch</code>	Créer un fichier vide	<code>touch fic1.txt</code>
<code>cp</code>	Copier (fichiers, répertoires)	<code>cp fic1.txt monrep/</code>
<code>mv</code>	Déplacer ou renommer	<code>mv fic1.txt fic2.txt</code>
<code>rm</code>	Supprimer (<code>-r</code> récursif) — définitif !	<code>rm -r monrep</code>
<code>echo</code>	Afficher un texte (<code>></code> écrase, <code>>></code> ajoute)	<code>echo Bonjour >> log.txt</code>
<code>cat</code>	Afficher le contenu d'un fichier	<code>cat monfic</code>
<code>nano</code>	Éditer un fichier (<code>Ctrl+O</code> sauver, <code>Ctrl+X</code> quitter)	<code>nano /etc/hosts</code>
<code>chown</code>	Changer le propriétaire (et le groupe avec <code>:</code>)	<code>chown www-data:www-data -R glpi</code>

Commande	Rôle	Exemple
<code>chgrp</code>	Changer le groupe propriétaire	<code>chgrp etudiants fic1.txt</code>
<code>chmod</code>	Changer les permissions (symbolique ou octal)	<code>chmod 755 monrep</code>
<code>apt</code>	Gérer les paquets (<code>update</code> , <code>upgrade</code> , <code>install</code>)	<code>apt install wget</code>
<code>wget</code>	Télécharger (<code>-P</code> dossier cible, <code>-O</code> renommer)	<code>wget -P /var/www url</code>
<code>tar</code>	Extraire une archive (<code>-x</code> extraire, <code>-v</code> verbeux, <code>-f</code> fichier)	<code>tar -xvf glpi-10.0.6.tgz</code>

Félicitations !

Vous êtes arrivé à la fin de la leçon sur Linux Debian. Vous disposez maintenant des fondamentaux pour administrer un serveur Debian en ligne de commande : ces commandes seront utilisées dans **tous** les TP Linux à venir (Apache, MariaDB, GLPI, Zabbix...).